Predicting the Root Cause of Flaky Tests Based on Test Smells

Jing Wang
College of Information Science and Technology
Beijing University of Chemical Technology
Beijing, China
2023200827@buct.edu.cn

Weixi Zhang
College of Information Science and Technology
Beijing University of Chemical Technology
Beijing, China
2021400216@buct.edu.cn

Weiwei Wang
College of Information Engineering
Beijing Institute of Petrochemical Technology
Beijing, China
wangweiwei@bipt.edu.cn

Ruilian Zhao
College of Information Science and Technology
Beijing University of Chemical Technology
Beijing, China
rlzhao@mail.buct.edu.cn

Ying Shang*
College of Information Science and Technology
Beijing University of Chemical Technology
Beijing, China
shangy@mail.buct.edu.cn

2025 IEEE/ACM 22nd International Conference on Software and Systems Reuse (ICSR) | 979-8-3315-2617-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSR66718.2025.00015

***Abstract*—Flaky tests refer to test cases that exhibit inconsistent behaviors across multiple executions, potentially passing or failing unpredictably. They are frequently associated with suboptimal design practices that testers may utilize when crafting test cases, which undermine the quality of software testing. So, identifying the root causes of flaky tests is crucial for fixing them. Currently, inspired by the success of the Large Language Models (LLMs), researchers leverage the pre-trained language model to embed flaky test code as vectors and predict its root cause category based on vector similarity measures. However, such code embeddings generated by LLM mainly focus on capturing general semantic features but lack sufficient comprehension of the behavioral patterns involved in test scenarios, resulting in poor root cause identification. Test smells, which reflect poor coding practices or habits when writing test cases, provide complementary information in the root cause identification of test flakiness. Therefore, this paper proposes a root cause identification method for flaky tests based on test smells. Test smells are used to abstract and express behavioral patterns of test codes, and general semantic features extracted by vector embeddings to enhance the feature representation of flaky tests. Furthermore, to capture the complex nonlinear relationships between test smell features and code embeddings, a Feedforward Neural Network is constructed to categorize the root cause of test flakiness. To validate the effectiveness of our method, we performed evaluations on a dataset consisting of 451 Java flaky test cases. The experimental results indicate that our method achieves an F1-score of 80%, which is 7% higher than that of the baseline model that does not incorporate test smells.**

***Index Terms*—flaky tests root causes, test smells, software testing, LLMs, CodeBERT**

I. Introduction

Software developers rely on tests to identify bugs in their code and provide signals about the correctness of the code [1]. If a test fails, developers often assume that this failure represents a recently introduced bug. However, a growing and worrying trend is the occurrence of regression test failures that are not attributable to recent code changes, but rather to flaky tests [2], which damage the credibility of software testing.

Flaky tests are inevitable in real-world test suites [3]. So flaky test detection techniques have received intensive attention in the research community. Many techniques require multiple runs of test cases to determine if they are flaky [4]–[6], which makes their deployment prohibitively costly in large-scale projects. To address this issue, recent studies have proposed machine learning techniques for predicting flaky test. These approaches use dynamic features such as code coverage and execution history [2], [7], static features such as Java keywords as vocabulary features [8], [9], and other features like code complexity and test smells [2], [7], [10] to predict flaky tests. However, the definitions of these features may not be exhaustive. Inspired by large language models, Flakify [11] leverages the pre-trained *CodeBERT* model and fine-tunes it to classify test cases as either flaky or non-flaky based on the test code. Notably, Flakify preprocesses the test code by filtering out sentences that contains test smells, which inevitably compromises the semantic context of the code to some extent.

Currently, most studies only predict whether a test is flaky, but fail to delve into the root cause of test flakiness. Identifying the root causes of flaky tests is crucial, as it can reveal deficiencies in test design or uncover latent defects within the software. Determining the root cause category that leads to flaky tests can help developers quickly focus on critical areas

979-8-3315-2617-7/25/$31.00 ©2025 IEEE
DOI 10.1109/ICSR66718.2025.00015

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.

of the code, thereby significantly improving the efficiency of resolving flaky test issues. FlakyCat [12] uses the pre-trained *CodeBERT* model to embed test code vectors and fine-tunes a Siamese few-shot learning network to predict the root cause category of flaky tests. However, FlakyCat only relies solely on the code embeddings generated by *CodeBERT*, which captures general semantic features and ignores the flakiness issues caused by poor coding practices when writing test code, thereby limiting the accuracy of root cause identification.

Suboptimal designs during test case development may lead to flaky tests. These include reliance on external resources or environmental states (such as databases, file systems, or network connections), lack of isolation, the introduction of time dependencies, and the failure to properly handle resource contention. Code smells represent patterns or structures in the code that suggest the presence of design flaws or poor coding practices [13]. Test code, similar to production code, much like production code, is susceptible to quality issues [14]. *Test smells* refer to design problems in test code and have been extensively studied. They are commonly present in both manually written and automatically generated test cases [15]. And it is observed that test smells, such as dependencies on external environments (e.g., database states), are closely related to test flakiness. So, an increase in test smells can serve as an indicator for detecting flaky tests [16]. Inspired by test smells, we propose test smells that are highly correlated with the root causes of flaky tests, aiming to provide strong guidance for the identification of the root causes of flaky tests.

Therefore, this paper proposes a root cause identification method for flaky tests that integrates test smells and code semantic features to enhance the code representation of flaky test. Furthermore, to capture the high-dimensional nonlinear relationships between test smell features and code embeddings, a Feedforward Neural Network is employed as a classifier to categorize the root causes of test flakiness. The major contributions of this paper are as follows:

1. This study identifies and formalizes six test smells associated with the root causes of flaky tests, three of which are proposed herein, along with corresponding detection methods developed through the analysis of behavioral patterns in the flaky test code.
2. A root cause identification method for flaky tests is proposed, which integrates test smells and code semantic features to enhance the code representation of flaky tests, and utilizes a Feedforward Neural Network to categorize the root causes of test flakiness.
3. To validate the effectiveness of our method, the evaluation is conducted on a dataset consisting of 451 Java flaky test cases. The experimental results indicate that our method achieves an F1-score of 80%, which is 7% higher than that of the baseline model that does not incorporate test smells.

The rest of this paper is organized as follows. Section II introduces the background of flaky test root causes and test smells. Section III defines the problem and provides a detailed introduction to the test smells associated with flaky test root causes that we have mined and defined. An overview of our proposed framework is also presented and its key components are described in detail. Section IV describes the evaluation methods, including datasets, baselines, implementation, and metrics. Section V presents the experimental results. Section VI discusses threats to validity. Section VII concludes the paper.

II. Background

A. Root Causes of Flaky Tests

Flaky tests refer to the phenomenon that test results are inconsistent across different runs on the same code and environment. To address this issue, testers generally need to identify the underlying causes of test flakiness in order to implement targeted solutions. Since multiple factors contribute to test flakiness, identifying the category of root causes of flaky tests can effectively guide testers to focus on critical areas and uncover potential issues.

Luo et al. [17] conducted a detailed analysis of 161 commits and classified the main causes of flaky tests by analyzing the list of the most prominent flaky test categories that developers and researchers need to pay attention to. Among these categories, the most common root causes are Async wait (45%), Concurrency (20%), and Test Order Dependencies (12%). These three causes account for 77% of the cases [17]. Additionally, Time and Unordered collections are found to be highly prevalent. Consequently, our study focuses on these five most common categories, as identifying them can greatly assist developers in fixing flaky tests. The five root causes of flaky tests, along with their descriptions, are detailed in Table I.

TABLE I: Flaky test root causes and descriptions

Root cause	**Description**
Async Wait	When the test execution makes an asynchronous call but does not properly wait for the result of the call to be available before using it.
Concurrency	When the test starts several threads interact non-deterministically.
Test Order Dependency	When the test results depend on the order in which the test cases are executed.
Time	When the test assertion depends on the system time.
Unordered Collections	When iterating over an unordered collection, the code assumes that the elements are returned in a specific order.

B. Test Smells

The concept of test smell was first introduced by Van Deursen et al. [18], who defined it as poor design choices implemented within the test code, specifically concerning the organization, implementation, and interaction of test cases. Test smells can diminish the effectiveness of test cases in detecting faults and hinder developers' ability to interact with

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.

the test code [19]. For instance, prior studies have shown that the results of certain test cases may be flaky due to poor practices in test code [20].

Existing studies have utilized test smells to address flaky tests. Alshammari et al. [2] employed eight test smells as features to predict test flakiness. Peruma et al. [21] introduced a tool called tsDetect to detect 19 types of test smells in JUnit-based unit tests. Camara et al. [10] applied this tool to detect test smells for predicting flaky test cases. These studies primarily focus on predicting whether a test case is flaky. Among the test smells used in these studies, some are related to the root causes of test flakiness, such as Mystery Guest, Fire and Forget, and Sleepy Test, which are also adopted in this work. However, many test smells are not directly associated with the underlying causes of flakiness, indicating a need for further exploration of more specific test smells linked to the root causes of flaky tests.

III. Methods

Flaky tests due to different root causes are often associated with corresponding poor coding practices(i.e., test smells). By focusing on these test smells, valuable clues can be provided for the automated identification of the root causes of flaky tests. However, relying solely on test smell features may overlook the complex behavior of the code in specific contexts. For this reason, we also utilize *CodeBERT* to extract the semantic features of flaky test code, supplementing the test smell features and collectively facilitating the automated identification of the root causes of flaky tests.

In this section, we first outline the definition of the problem. Then, we detail the overall framework of our method. Finally, each module of the framework is illustrated.

A. Problem Definition

The task of identifying the root cause category of flaky tests is defined as a multi-class classification problem based solely on the flaky test code. Formally, the dataset is represented as:

$$\{(t_i, y_i) \mid t_i \in T, y_i \in Y\}_{i=1}^{N}, \tag{1}$$

where:

- T is the set of flaky test samples.
- $Y = \{1, \ldots, k\}$ is the set of root cause categories, where each y_i represents a specific root cause of test flakiness. More concretely, the categories are listed in Table I.
- N is the total number of samples, and k is the number of root cause categories.

To achieve this, we extract two types of features for each test t_i:

- *Test smell features*: A set of characteristics that are closely related to the root causes of test flakiness, represented by binary features.
- *Semantic features*: The high-dimensional vector representations of flaky test code generated by *CodeBERT*.

The feature set x_i for a flaky test t_i is constructed by combining its test smell features and semantic features. Using this fused representation, our objective is to learn a mapping function $f\colon X \to Y$, where X denotes the fused feature space. This function f is optimized by minimizing a loss function, defined as:

$$\min \sum_{i=1}^{N} \mathcal{L}(f(x_i), y_i), \tag{2}$$

where $\mathcal{L}(\cdot)$ is the cross-entropy loss function.

B. The Overall Framework

In this section, we introduce the overall framework of our approach. Our core goal is to predict the category of the root cause of flaky tests based on test smells and code semantics. Concretely, a set of test smells that are highly correlated with flaky test root causes are first defined based on the code characteristic. On this basis, the test smell features are extracted by our *test smell detector* which we developed to parse the test code and identify test smells via pattern matching. Secondly, the semantic features of flaky tests are extracted using the sequence embedding methods via pre-trained large language model *CodeBERT* [22]. Subsequently, the Feed Forward Neural network is employed to classify flaky tests based on the fused features of test smell features and semantic features, with each category corresponding to a root cause category.

The overall working framework is shown in Fig. 1. Each module of our method is illustrated in detail in the subsequent parts of this section.

C. Root Causes of Flaky Tests

In this section, we analyze the bad coding practices of flaky tests arising from different root causes.

The root cause of flaky tests is classified as *Async Wait* [17] when the test execution involves an asynchronous call but fails to properly await the result before utilizing it. Among the flaky tests identified as Async Wait by Luo et al. [17], 34% use `sleep` or `waitFor` with a time delay to wait for the call to complete. However, the actual completion time of the operation can vary due to factors such as system load and resource contention. For instance, on slower machines, inadequate waiting time may not ensure the operation's completion, leading to intermittent test failures. The use of sleep operation cannot guarantee the required sequence of events during execution, rendering the tests unreliable and flaky.

Existing machine learning methods for predicting the root causes of flaky tests face challenges in distinguishing between the *Async Wait* and *Concurrency* categories due to their overlapping features, often leading to misclassification. It is noteworthy that the primary distinction between the *Async Wait* and *Concurrency* categories lies in the fact that *Async Wait* explicitly involves synchronization with external or remote resources [3], whereas most flaky tests in the *Concurrency* category pertain to concurrent access to memory resources rather than external resources [17].

The critical feature of the *Async Wait* and *Concurrency* categories lies in the use of multithreading mechanisms, which

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.

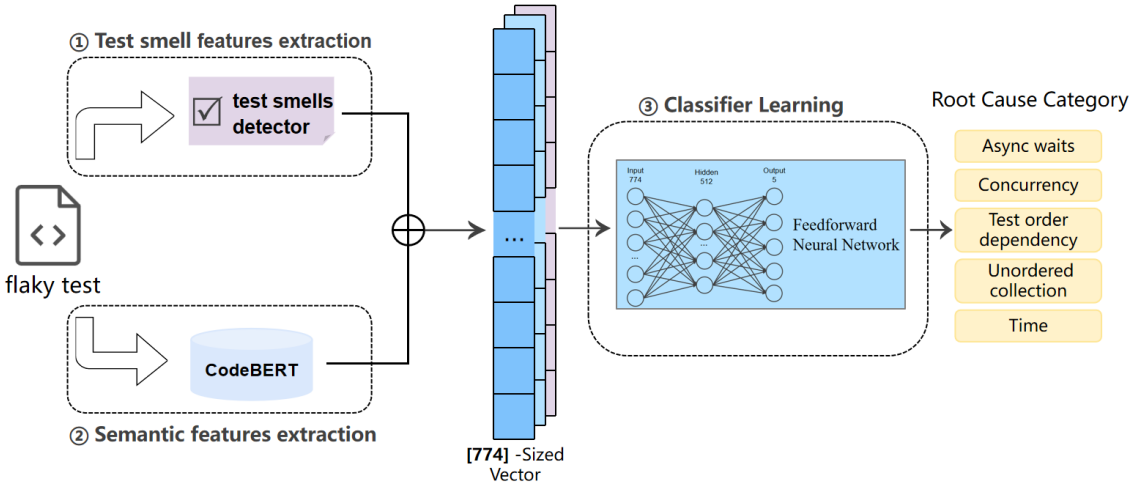


Fig. 1: Overall framework

clearly differentiates them from other categories. Therefore, in identifying the root causes of flaky tests, it is crucial to detect whether multithreading mechanisms are involved in the test implementation.

When flakiness in a test arises from different threads interacting in a bad way (excluding the case of asynchronous calls which is common in the *Async Wait* category), the root cause is classified as *Concurrency* [17]. It has been observed that terms such as `thread` and `threading` can be indicators of abnormal concurrent testing [3].

Relying on system time can result in non-deterministic failures [3]. When tests occasionally fail due to time-related issues, identifying and resolving the root cause can be challenging, thereby increasing the difficulty of debugging and maintenance. For instance, a test may fail when the UTC time zone changes at midnight. Additionally, some tests may fail due to discrepancies in time reporting accuracy across different platforms. To mitigate such issues, best practices discourage the direct use of system time in code design.

The flaky test root cause is classified as *Test Order Dependency* when the test outcome depends on the order in which the tests are run [17]. This issue arises when tests rely on shared states that are either improperly set up or inadequately cleaned, with these shared states potentially residing in main memory or external resources.

Flakiness caused by *Unordered Collections* typically occurs when iterating over an unordered collection (such as a set), where the code assumes that the elements are returned in a specific order. Since different executions may produce different orders, such assumptions can lead to test flaky.

D. Exploration of Test Smells Association with Flaky Test Root Causes

As mentioned above, flaky tests are related to suboptimal designs adopted by testers when writing test cases. *Test smells* can help capture design issues in test code, which are commonly observed in both manually written and automatically generated test cases. Zolfaghari et al. [16] found that an increase in test smells can serve as an indicator for detecting flaky tests. Through analysis, we find six test smells are associated with flaky test root causes, which are listed in Table II.

TABLE II: Test smells and their description

Test Smell	Description
Mystery Guest [2]	A test accesses external resources such as file systems, databases, networks, clusters, etc.
Sleepy Test [21]	A test involves the SLEEP method on a thread.
Fire and Forget [2]	A test launches background threads or processes.
MultiThreading	A test involves thread synchronization and scheduling operations in a multi-threaded environment.
Time Acquisition	A test involves retrieving the local or system time.
Ordered Hypothesis	A test assertion involves implicit assumptions about the ordering of the unordered collection.

Each row corresponds to a specific test smell, with the Description detailing its characteristics. Among these, the first three test smells are defined in existing research, while the remaining three are proposed by us. The test smells we propose are essential for identifying specific causes of test flakiness. Test flakiness caused by *Concurrency* often involves multithreading mechanisms, and the *MultiThreading* test smell plays a crucial role in identifying issues related to concurrency. The *Time Acquisition* test smell focuses on whether the test accesses system time, and it is instrumental in uncovering flakiness caused by time-related factors. The *Ordered Assumption* test smell occurs when there is an assumption of order in unordered collections during assertions, thereby facilitating the identification of flaky tests associated with *Unordered Collections*.

E. Test Smell Features Extraction

The presence of these test smells exposes potential issues underlying test flakiness, such as external resource calls and

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.

implicit assumptions, which can provide crucial clues into the root causes identification of flaky tests. We extract test smell features from test code, where the smells are described in Table II. Each test smell feature is represented as a binary value, with `1` denoting the presence and `0` the absence of a specific test smell.

To detect test smells in Java flaky tests, similar to Flakify, we adopt a heuristic approach based on Abstract Syntax Tree (AST) [23] parsing. We parse flaky test code into an AST and use node pattern matching within the AST structure to identify the presence of specific test smells. The heuristic rules for detecting test smells are as follows:

- *Mystery Guest:* A statement invokes a method that accesses external resources, such as the file system (via java.io.File), database system (via java.sql, javax.sql, or javax.persistence), or network (via java.net or javax.net) [11]; A statement invokes methods or accesses classes whose names contain keywords such as `file`, `Database`, `Hbase`, `Cassandra`, `table`, `Transport`, `service`, `Servlet`, `Server`, `Message`, `Session`, `Connection`, `Broadcast`, `Client`, `subscribe`, `Cluster`, and others.
- *Sleepy Test:* A statement invokes the `Thread.sleep()` method; A test invokes a method with a specific time delay, where the method name contains the term `wait`.
- *Fire and Forget:* A statement invokes a method that belongs to the `java.lang.Thread` class, `java.lang.Runnable` interface, or `java.util.concurrent` package [11]; A statement invokes a method that belongs to the class whose name contains `thread`; A statement invokes a method whose name includes the term `thread`.
- *MultiThreading:* A test involves thread synchronization and scheduling operations in a multi-threaded environment. This includes: Classes such as `CountDownLatch`, `ExecutorService`, `ScheduledExecutorService`, `Lock`; Classes whose names contain `Atomic`, such as `AtomicInteger`, which provide atomic operations in a multi-threaded environment; Classes whose names contain `Executor` or `Scheduler`, such as `ScheduledExecutorService`, which provide task execution and scheduling mechanisms in a multi-threaded environment.
- *Time Acquisition:* A statement invokes a method or accesses a class or variable related to time manipulation or acquisition. This includes:
  - Method such as `currentTimeMillis()`, `now()`, `NanoTime()`, `getTime()`, `getDurationMillis()`, `getMinutes()`, `getSeconds()`, `getMillis()`, `between()`, etc.; Methods that contain any of the following strings: `currentTime`, `getTime`;
  - Class such as `LocalDate`, `LocalTime`, `LocalDateTime`, `DateTime`, `Duration`, `Period`;
  - Variable whose name contains `stamp`, `epoch`, `startTime`, `endTime`.
- *Ordered Hypothesis:* A statement invokes a method whose name contains `assert` or `verify`, and involves the `encode()` method or data structures such as `list`, `string`, `map`, or key-value pairs, where the order of elements may be implicitly assumed.

F. Semantic Features Extraction

We extract test code embeddings using *CodeBERT*. *CodeBERT* is pre-trained on a large-scale corpus of code, allowing it to generate high-quality representations of code. Compared to other language models, *CodeBERT* leverages six programming languages for natural language and programming language *(NL-PL)* matching during training, making it particularly suitable for code-related tasks. The process of generating code embeddings with *CodeBERT* is depicted in Fig. 2 and elaborated upon below:

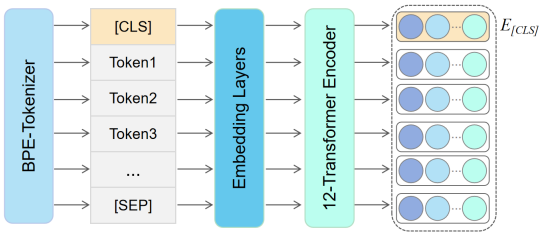


Fig. 2: *CodeBERT*-based code embedding process

The process begins with tokenization, where the flaky test is segmented into subwords using the *CodeBERT tokenizer*. This tokenizer, based on *BERT*'s tokenization method and *Byte-Pair Encoding (BPE)* technology, is designed to handle the grammatical structure and unique identifiers of various programming languages. It recognizes keywords, operators, and symbols as distinct tokens while decomposing uncommon identifiers or lengthy variable names into more frequent subwords. Special tokens *[CLS]* and *[SEP]* are added to the sequence to denote its boundaries.

$$\textit{BPE-Tokenizer}(t_i) = \{\textit{CLS}, \textit{Token}_1, \textit{Token}_2, \textit{Token}_3, \ldots, \textit{Token}_n, \textit{SEP}\} \tag{3}$$

where $\textit{Token}_1, \textit{Token}_2, \ldots, \textit{Token}_n$ are the subwords obtained through the Byte-Pair Encoding method applied to the test input t_i.

After tokenization, the segmented input is converted into a series of embeddings. These include *Token Embeddings*, derived from the token indices in the vocabulary; *Segment Embeddings*, which differentiate between distinct segments of the input; and *Position Embeddings*, which provide positional information for each token. These embeddings are combined to form a rich representation of the input, capturing its syntactic, semantic, and positional features [22].

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.

The combined embeddings are then fed into *CodeBERT's* 12-layer Transformer-Encoder architecture. Each layer employs self-attention mechanisms and feedforward networks, with residual connections and normalization to ensure stable training and effective representation learning. Through this process, each token is transformed into a 768-dimensional vector, The *[CLS]* token is mapped to vector $E_{[CLS]}$, which encapsulates the overall semantic, syntactic, and contextual information of the code. Consequently, we utilize $E_{[CLS]}$ as semantic features for the flaky test.

Since the *CodeBERT* model's input sequence length is limited to 512 tokens, for flaky test code snippets exceeding this limit, we partition the sequence into segments and apply average pooling to the embeddings of each segment. These pooled embeddings are then used as the semantic features.

G. Classification Learning

We integrate the test smell features with semantic features to enhance the flaky test code representation. This fusion method not only leverages the powerful semantic representation capabilities of *CodeBERT* but also incorporates the factors of test flakiness revealed by the test smell features to provide more comprehensive feature inputs for the classification model. In this way, we are able to more accurately capture the factors in the code that contribute to test flakiness and effectively predict the root causes.

After feature extraction and fusion, we chose a Feedforward Neural Network as the classification model. The reason for choosing FNN as the classification model is its powerful feature fusion capability and advantage in processing structured data. The *CodeBERT* embedding vector provides deep semantic information about the code, while the test smell features reflect the underlying factors of test flakiness. Through the multi-layer structure of FNN, we can effectively fuse these features, capture the complex relationships among them, and achieve efficient classification.

The FNN model consists of multiple fully connected layers, with the input layer receiving the concatenated vector of the 6-dimensional test smell features and the 768-dimensional semantic feature from *CodeBERT*. The hidden layer contains 512 neurons and uses the ReLU (Rectified Linear Unit) activation function to process the features through linear transformation and nonlinear processing to learn the complex relationship between features. To prevent the model from overfitting, we added a dropout layer. The output layer uses the Softmax activation function to convert the results of the hidden layer into the probability distribution of flaky categories. To optimize the model parameters, we chose the cross entropy loss function and the Adam optimizer. The cross-entropy loss function is suitable for multi-classification problems, while the Adam optimizer can effectively adjust the learning rate and accelerate the training process.

IV. Experiments

A. Research Questions

- **RQ1: How effective is the proposed method in predicting the root cause category of flaky tests? Which classifier can achieve the best performance?**

In this paper, a root cause identification method is proposed for flaky tests that integrates test smells and code semantic features to enhance the code representation of flaky test code, and a Feedforward Neural Network is utilized to classify the root causes of test flakiness. By investigating this RQ1, we aim to explore the effectiveness of integrating these features and identify the classifier that performs best in root cause identification for flaky tests.

- **RQ2: How effective are the test smells related to the root causes of test flakiness proposed in this paper?**

Existing test smell detection tools are available, but not all test smells are directly or indirectly related to the root causes of test flakiness. This research question RQ2 aims to assess, through comparison with existing tools, whether the test smells specifically associated with flaky test root causes proposed in this paper can more accurately guide the identification of root causes.

- **RQ3: How effective is our approach in predicting each individual category of flaky test root causes?**

By investigating this RQ3, we aim to evaluate the classification capabilities of our approach across different root cause categories of flaky tests.

- **RQ4: What do test smell features contribute to the output values of the Feedforward Neural Network model for classifying flaky tests?**

In the interest of model explainability, the Shapely Additive Explanations (SHAP) technique is applied to investigate the impact of each individual feature within the fused features. By answering this RQ4, we aim to evaluate whether the test smell features have an important contribution to the model and the influence of each test smell on the model's decision for each category.

B. Experiment Setup

1) Dataset: We use the Flakycat [12] dataset, which contains 964 flaky tests distributed in 13 different categories. Specifically, the dataset consists of 451 original flaky test samples, along with additional samples generated through oversampling via the application of basic perturbations.

Due to the limited number of samples in certain categories, consistent with the practice adopted in Flakycat, we focus on the five most common unstable test categories: *Asynchronous Waiting*, *Concurrency*, *Test Order Dependency*, *Unordered Collections*, and *Time*, with other categories having fewer than 30 original samples.

2) Baseline methods: We utilize the existing method FlakyCat [12] as a baseline. FlakyCat employs *CodeBERT* for code representation and leverages Siamese networks to train a multi-class classifier, which categorizes flaky tests based on their root causes.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.

In addition, we also use different combinations of code representation methods (including and excluding test smell features) and four machine learning classifiers (support vector machine, decision tree, k-nearest neighbor algorithm, random forest) as comparison methods.

3) Implementation: We implemented our model using PyTorch, an opensource deep learning framework. Our experiments were performed with Ubuntu 20.04.3.

The neural networks are trained in a batch-wise fashion until converging and the batch size is set to 32. The dimension of the vector representation of each node is set to 774 and the dropout is set to 0.2. ADAM optimization algorithm is used to train the model with the learning rate of 0.001.

C. Evaluation Metrics

We used the following evaluation metrics to measure the effectiveness of our model:

Accuracy evaluates the performance by determining how many instances can be correctly labeled. It is calculated as:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}, \tag{4}$$

Precision is the fraction of true positives among the detected ones. It is defined as:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{5}$$

Recall measures how many positives can be correctly detected. It is calculated as:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{6}$$

Among them, TP (true positive) is the instance correctly predicted as positive, TN (true negative) is the instance incorrectly predicted as negative, FP (false positive) is the instance incorrectly predicted as positive, and FN (false negative) is the instance incorrectly predicted as negative.

F1 score is the harmonic mean of Recall and Precision, and can be calculated as:

$$\text{F1 score} = 2 \times \frac{\text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}, \tag{7}$$

MCC (Matthews Correlation Coefficient) is a performance metric used to evaluate the effectiveness of multi-class classification models, particularly suitable for imbalanced datasets. For a multi-class classification problem, MCC can be calculated using the confusion matrix as:

$$\text{MCC} = \frac{\sum_k \sum_l \sum_m C_{kk} C_{lm} - C_{kl} C_{mk}}{\sqrt{(\sum_k S_k)(\sum_k P_k)(\sum_k T_k)(\sum_k N_k)}} \tag{8}$$

where: - C_{ij} denotes the element in the confusion matrix representing the number of samples with the true label i predicted as class j. - C_{kk} is the number of correctly classified samples for class k (diagonal elements). - $S_k = \sum_j C_{kj}$ represents the total number of samples in class k. - $P_k = \sum_i C_{ik}$ represents the total number of samples predicted as class k. - $T_k = S_k - C_{kk}$ is the number of samples truly belonging to class k but misclassified. - $N_k = \text{total samples} - P_k$ is the number of samples not belonging to class k but misclassified as class k.

Finally, *AUC* (Area Under the Curve) is the area under the ROC curve (Receiver Operating Characteristic Curve), which is used to evaluate the performance of a classification model. We use a weighted metric to accommodate our imbalanced data set.

V. Results

A. *RQ1: How effective is the proposed method in predicting the root cause category of flaky tests? Which classifier can achieve the best performance?*

To evaluate the effectiveness of the proposed method, we conducted a series of experiments comparing the performance of flaky test root cause identification using only semantic features versus the fusion of semantic features and test smell features across various classifiers. The baseline method, FlakyCat, uses a combination of only semantic features and a Siamese network based on similarity. Additionally, we included multiple classifier types for evaluation, including machine learning models and the feed-forward neural networks employed in this study.

In our experiments, the dataset was divided into training and testing sets with a 3:1 split ratio. To ensure that the model is evaluated on completely unseen data, the generated samples and their corresponding original samples were consistently allocated to either the training set or the testing set. We use a 4-fold cross-validation on the training data to select and evaluate the best model. The results of the comparison are summarized in Table III.

The results in Table III demonstrate the effectiveness of integrating test smells and code semantic features in identifying the root cause category of flaky tests. The inclusion of test smell features improved classification performance across all evaluated models. This suggests that semantic features alone may be insufficient for accurately representing the underlying flakiness. Test smell features are critical information for the flakiness of flaky tests, and their integration with semantic features can enhance the accuracy of identifying the root cause category of flaky tests.

Among the classifiers evaluated, the feed-forward neural network (FNN) showed superior performance when both semantic and test smell features were utilized. Specifically, the FNN achieved the highest accuracy (0.81), precision (0.81), and F1 score (0.80), making it the most effective classifier in this study. Traditional machine learning models like Random Forest (RF) and k-Nearest Neighbor (KNN) also showed notable improvements with the addition of test smell features, but their performance was marginally less effective than that of the FNN.

Answer to RQ1: Compared to using only semantic features, incorporating test smell features alongside semantic features significantly enhances the model's ability to predict the root

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Comparison of classification performance **without (O)** and **with (✓)** test smells across different classifiers.

Model Configuration	SVM	SVM	DT	DT	KNN	KNN	RF	RF	Siamese	Siamese	FNN	FNN
	O	✓	O	✓	O	✓	O	✓	O(FlakyCat)	✓	O	✓
Accuracy	0.38	0.59	0.46	0.60	0.57	0.69	0.66	0.71	0.73	0.79	0.70	0.81
Precision	0.24	0.45	0.46	0.61	0.59	0.68	0.67	0.71	0.74	0.79	0.71	0.81
Recall	0.38	0.59	0.46	0.60	0.57	0.69	0.66	0.71	0.73	0.79	0.70	0.81
F1 Score	0.29	0.50	0.46	0.60	0.56	0.68	0.65	0.69	0.73	0.78	0.70	0.80
MCC	0.13	0.46	0.30	0.48	0.43	0.59	0.55	0.62	0.65	0.72	0.61	0.75
AUC	0.55	0.71	0.65	0.74	0.71	0.79	0.76	0.80	0.83	0.86	0.90	0.93

cause category of flaky tests. This demonstrates the effectiveness of using the fused features of test smells and semantic feature for identifying the root causes of flaky tests. Based on the fused features, the feed-forward neural network proposed in this study emerges as the most effective classifier.

*B. **RQ2: How effective are the test smells directly linked to the root causes of test flakiness proposed in this paper in flaky test root cause identification?***

tsDetect [21] is a widely recognized automated test smell detection tool for Java software systems, capable of identifying 19 different test smells in test code. However, not all of these test smells are associated with test flakiness root causes. The test smell detection approach proposed in this paper, on the other hand, specifically targets six test smells that are closely related to flaky root causes.

To answer this RQ, we applied both our method and the tsDetect tool to the test code to extract the corresponding test smell features. These extracted features were then combined with semantic features and used to build multiple classification models, including traditional machine learning models, a Siamese network model, and Feedforward Neural Network (FNN) models. The performance of predicting flaky test root cause category under different configuration combinations is shown in Table IV.

The results demonstrate that our test smell features consistently outperform those detected by tsDetect across all classifiers. This is evident in the higher accuracy, precision, recall, F1 score, MCC, and AUC values achieved by models using our test smell features. Especially in the Feedforward Neural Network model, when test smell features proposed in this paper are combined with semantic features, the model's F1 score is as high as 0.80, while the F1 score of the model is only 0.67 when the extensive test smell features detected by tsDetect are combined with semantic features.

Answer to RQ2: The introduction of six test smells specifically associated with flaky tests significantly outperforms the test smells detected by tsDetect in the task of flaky test root cause category prediction. This finding validates that the test smells proposed in this paper, which are tailored to flaky tests, play a crucial role in enhancing the accuracy of flaky test root cause prediction.

*C. **RQ3: How effective is our approach in predicting each category of flaky test root causes?***

Table V shows the prediction performance for different root cause categories, using only semantic features and employing the Feedforward Neural Network as the classifier.

The performance for different categories using test smell features and Semantic Features on Feedforward Neural Network are shown in Table VI.

Adding test smell features significantly improves the model's performance in predicting most categories. For example, the F1 score for the *Async wait* category improved from 0.65 to 0.82, etc. These improvements demonstrate the importance of test smell features in providing flaky root causes information that semantic features alone cannot capture.

However, the *Concurrency* category did not significantly benefit from the introduction of the test smell features, as its F1 score decreased slightly from 0.45 to 0.41. We hypothesize that this is due to the common code patterns in flaky tests caused by *Concurrency*, such as *Fire and Forget* and *Multithreading*, which are also common in flaky tests caused by *Async wait*.

Answer to RQ3: The performance enhancement in most categories confirms that our approach is more effective in predicting the root causes of flaky tests, especially in the async-wait and time categories.

*D. **RQ4: What contribution do test smell features have to the output values of the Feedforward Neural Network model for classifying flaky tests?***

To address this RQ, we employed the Shapley Additive Explanations (SHAP) technique [24]. SHAP utilizes concepts from game theory to quantify the contribution of each feature to the model output for a single data point across different categories. In our classification task, SHAP takes a feature matrix and a trained model and returns a three-dimensional matrix of SHAP values. The shape of this matrix is (number of samples, number of features, number of categories). The SHAP value at position (i, j, k) in the matrix represents the contribution of feature j to the model's output for sample i in category k, relative to the average output value of the dataset. By analyzing the SHAP values, we can gain insights into the influence of each feature on the model's predictions across different categories. We visualized the distribution of feature importance within each root cause category by generating SHAP summary plots. Fig. 3 displays the SHAP values under each root cause category as a beeswarm plot.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.

TABLE IV: Comparison of classification performance employing **tsDetect**'s (**tsD**) test smells and test smells proposed in this paper(**Ours**) across different classifiers.

Model Configuration	**SVM**	**SVM**	**DT**	**DT**	**KNN**	**KNN**	**RF**	**RF**	**Siamese**	**Siamese**	**FNN**	**FNN**
	tsD	**Ours**	**tsD**	**Ours**	**tsD**	**Ours**	**tsD**	**Ours**	**tsD**	**Ours**	**tsD**	**Ours**
Accuracy	0.39	0.59	0.45	0.60	0.46	0.69	0.66	0.71	0.72	0.79	0.66	0.81
Precision	0.25	0.45	0.45	0.61	0.46	0.68	0.68	0.71	0.77	0.79	0.71	0.81
Recall	0.39	0.59	0.45	0.60	0.46	0.69	0.66	0.71	0.72	0.79	0.66	0.81
F1 Score	0.30	0.50	0.45	0.60	0.44	0.68	0.63	0.69	0.72	0.78	0.67	0.80
MCC	0.14	0.46	0.28	0.48	0.27	0.59	0.54	0.62	0.65	0.72	0.58	0.75
AUC	0.56	0.71	0.64	0.74	0.63	0.79	0.76	0.80	0.82	0.86	0.89	0.93

TABLE V: Prediction performance for each category using only semantic features across Feedforward Neural Network

Category	Precision	Recall	F1 Score
Async Wait	0.67	0.64	0.65
Concurrency	0.41	0.50	0.45
Test Order Dependency	0.80	0.77	0.79
Unordered Collections	0.74	0.78	0.76
Time	0.88	0.79	0.83

TABLE VI: Prediction performance for each category using fused features across Feedforward Neural Network

Category	Precision	Recall	F1 Score
Async Wait	0.72	0.96	0.82
Concurrency	0.53	0.33	0.41
Test Order Dependency	0.92	0.84	0.88
Unordered Collections	0.96	0.69	0.81
Time	0.90	1.00	0.95

As shown in Fig. 3, the six test smell features we defined consistently rank among the top six in contribution across the classification task, indicating their significant impact on the model's decision-making process. In particular, Fig. 3a demonstrates that when the values of the *Sleepy Test* and *Mystery Guest* features are high (i.e., true), they positively contribute to the prediction of the *Async Wait* category. Conversely, when their values are low (i.e., false), these features exhibit negative contributions. This indicates that the *Sleepy Test* and *Mystery Guest* features are critical in accurately predicting the *Async Wait* category. Fig. 3b illustrates that when the *Fire and Forget* test smell is high(i.e., true), it positively influences the prediction of the *concurrency* category. This finding underscores that a key characteristic of flaky tests caused by *concurrency* issues is the involvement of thread-related operations. Fig. 3d shows that when the *Ordered Hypothesis* test smell is high (i.e. true), they tend to make positive contributions to predicting the *Unordered Collections* category, which confirms that an important feature of flaky tests caused by *Unordered Collections* is the order assumption on unordered sets. Fig. 3e demonstrates that when the *Time Acquisition* test smell is high (i.e., true), it tends to positively influence the prediction of the *Time* category. This indicates that flaky tests associated with the *Time* category are often linked to operations involving time acquisition.

Answer to RQ4: The six test smell features we defined consistently rank among the top six in terms of their contribution to the classification of flaky test root causes, exerting a significant influence on the model's decision-making process.

VI. Threats to Validity

In this section, we discuss several potential threats to validity.

Regarding internal validity, one potential threat is selection bias introduced by the dataset used in this study. The dataset may not fully capture the diversity of flaky tests encountered in real-world software projects, particularly regarding the complexity and scope of their root causes. Furthermore, the manual labeling of flaky tests might introduce human error or subjectivity, which could affect the accuracy of the model.

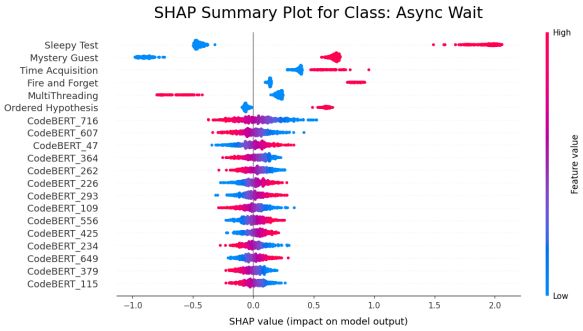
Regarding external validity, we focus on the five most common root cause categories of flaky tests in Java, such as *Concurrency* and *Async Wait*. We defined a set of test smells that are closely related to 5 types of root causes of flaky tests, provided relatively comprehensive detection methods for identifying them, and trained a model to identify flaky tests within these five categories. However, the model's performance may vary when applied to other root cause categories.

To address these threats, future research will broaden the dataset to encompass a wider variety of flaky tests caused by other root causes, as well as correspondingly investigate additional flaky test smells.

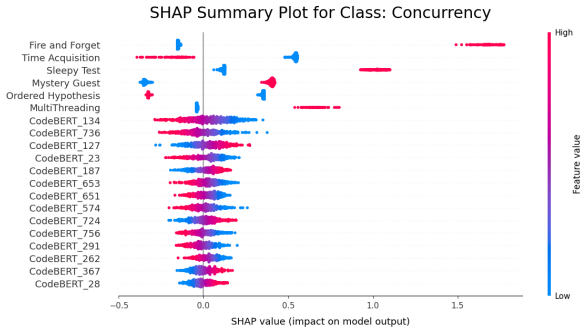
VII. Conclusion

In this paper, we propose a method to predict the root cause category of flaky tests based on test smells and semantic features. A set of test smells that related to flaky tests' root causes are defined and formulated. By integrating test smell features with the semantic features derived from *CodeBERT*, we can extract key characteristics of the root causes of test flakiness from the flaky test code. Then a Feedforward Neural Network is employed to perform deep learning on the fused features to achieve an effective classification of the root cause of the flaky test. The experimental evaluation demonstrates that test smells can significantly improve the classification of flaky tests, achieving an 80% F1 score. In addition, through the analysis of the SHAP value of each individual feature, we further evaluated the contribution of each feature. We observed

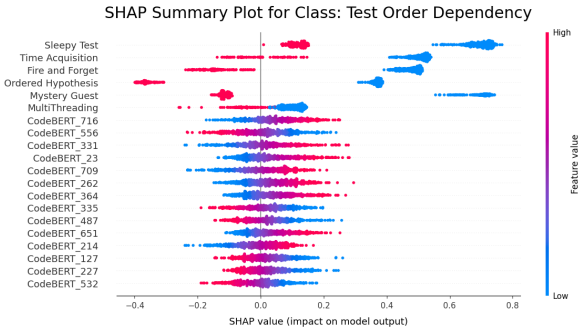
Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.



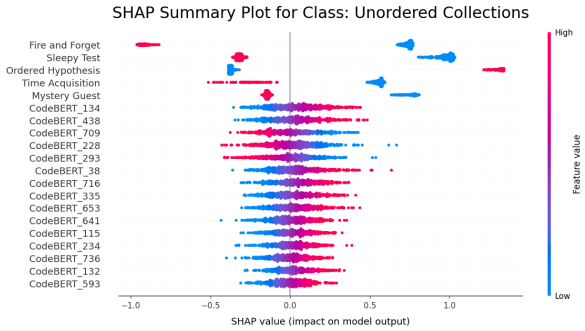
(a) SHAP Summary Plot for Async Wait



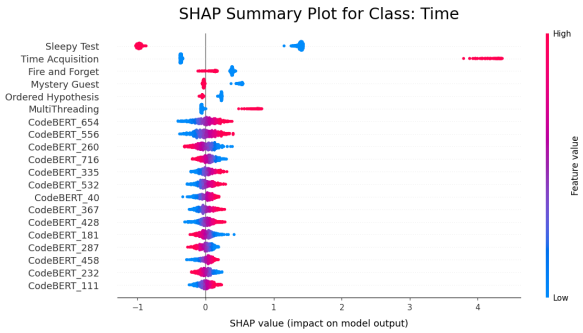
(b) SHAP Summary Plot for Concurrency



(c) SHAP Summary Plot for Test Order Dependency



(d) SHAP Summary Plot for Unordered Collections



(e) SHAP Summary Plot for Time

Fig. 3: SHAP values as beeswarm plots. Blue points represent lower eigenvalues and red points represent higher eigenvalues.

that test smell features substantially impact model decision-making, with each test smell exhibiting significant positive or negative effects in specific categories. Besides, compared with other classification models, the performance of Feedforward Neural Network outperforms the Siamese Network approach based on code similarity measurement and traditional machine learning methods.

In the future, we plan to expand our flaky test dataset with more data and extend our set of test smells, along with their corresponding detection methods, thereby improving the generalizability of our approach.

VIII. Acknowledgment

The work described in this paper is supported by the National Natural Science Foundation of China under grant no. 62077003.

References

[1] G. M. Kapfhammer, "Software testing," *The Computer Science Handbook*, pp. 105–1, 2004.

[2] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "Flakeflagger: Predicting flakiness without rerunning tests," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1572–1584.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.

[3] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–74, 2021.

[4] J. Bell, G. Kaiser, E. Melski, and M. Dattatreya, "Efficient dependency detection for safe java test acceleration," in *Proceedings of the 2015 10th joint meeting on foundations of software engineering*, 2015, pp. 770–781.

[5] S. Dutta, A. Shi, R. Choudhary, Z. Zhang, A. Jain, and S. Misailovic, "Detecting flaky tests in probabilistic and machine learning applications," in *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*, 2020, pp. 211–224.

[6] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "idflakies: A framework for detecting and partially classifying flaky tests," in *2019 12th ieee conference on software testing, validation and verification (icst)*. IEEE, 2019, pp. 312–322.

[7] V. Pontillo, F. Palomba, and F. Ferrucci, "Toward static test flakiness prediction: A feasibility study," in *Proceedings of the 5th International Workshop on Machine Learning Techniques for Software Quality Evolution*, 2021, pp. 19–24.

[8] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 492–502.

[9] G. Haben, S. Habchi, M. Papadakis, M. Cordy, and Y. Le Traon, "A replication study on the usability of code vocabulary in predicting flaky tests," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 2021, pp. 219–229.

[10] B. Camara, M. Silva, A. Endo, and S. Vergilio, "On the use of test smells for prediction of flaky tests," in *Proceedings of the 6th Brazilian Symposium on Systematic and Automated Software Testing*, 2021, pp. 46–54.

[11] S. Fatima, T. A. Ghaleb, and L. Briand, "Flakify: A black-box, language model-based predictor for flaky tests," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1912–1927, 2022.

[12] A. Akli, G. Haben, S. Habchi, M. Papadakis, and Y. Le Traon, "Flakycat: Predicting flaky tests categories using few-shot learning," in *2023 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, 2023, pp. 140–151.

[13] A. Mori, A. C. Paiva, and S. R. Souza, "An approach to regression testing selection based on code changes and smells," in *Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing*, 2023, pp. 25–34.

[14] D. J. Kim, "An empirical study on the evolution of test smell," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*, 2020, pp. 149–151.

[15] A. Panichella, S. Panichella, G. Fraser, A. A. Sawant, and V. J. Hellendoorn, "Test smells 20 years later: detectability, validity, and reliability," *Empir. Softw. Eng.*, vol. 27, no. 7, p. 170, 2022. [Online]. Available: https://doi.org/10.1007/s10664-022-10207-5

[16] B. Zolfaghari, R. M. Parizi, G. Srivastava, and Y. Hailemariam, "Root causing, detecting, and fixing flaky tests: State of the art and future roadmap," *Software: Practice and Experience*, vol. 51, no. 5, pp. 851–867, 2021.

[17] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, 2014, pp. 643–653.

[18] A. Van Deursen, L. Moonen, A. Van Den Bergh, and G. Kok, "Refactoring test code," in *Proceedings of the 2nd international conference on extreme programming and flexible processes in software engineering (XP2001)*. Citeseer, 2001, pp. 92–95.

[19] V. G. Yusifoğlu, Y. Amannejad, and A. B. Can, "Software test-code engineering: A systematic mapping," *Information and Software Technology*, vol. 58, pp. 123–147, 2015.

[20] A. Vahabzadeh, A. M. Fard, and A. Mesbah, "An empirical study of bugs in test code," in *2015 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 2015, pp. 101–110.

[21] A. Peruma, K. Almalki, C. D. Newman, M. W. Mkaouer, A. Ouni, and F. Palomba, "Tsdetect: An open source test smells detection tool," in *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2020, pp. 1650–1654.

[22] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, and et al., "Codebert: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020, pp. 1536–1547.

[23] R. E. Noonan, "An algorithm for generating abstract syntax trees," *Computer Languages*, vol. 10, no. 3-4, pp. 225–236, 1985.

[24] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable ai for trees," *Nature machine intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 18:13:16 UTC from IEEE Xplore. Restrictions apply.